

C++ Pointers - 20 Questions and Answers

Question 1: Pointer Relationship

Question: Consider the following C++ code snippet:

```
cpp
int x = 10;
int* ptr1 = &x;
int* ptr2 = ptr1;
```

What is the relationship between ptr1 and ptr2 in this code?

Options:

- They are pointing to the same memory location.
- ptr1 is a copy of ptr2
- They are unrelated and point to different memory locations.
- ptr2 is a reference to ptr1

Answer: ☒ They are pointing to the same memory location.

Explanation:

- `ptr1` points to `x`
 - `ptr2` is assigned `ptr1`, so it also stores the same address as `ptr1`
 - Both pointers point to the same memory location (the variable `x`)
 - Changing `*ptr1` or `*ptr2` will change the value of `x`
-

Question 2: Null Pointers

Question: Which of the following statements is true about null pointers in C++?

Options:

- Null pointers always point to memory addresses containing zeros.
- Null pointers are the same as uninitialized pointers.
- Null pointers are represented by the value 0 or the `nullptr` keyword.
- Null pointers cannot be used in C++.

Answer:  Null pointers are represented by the value 0 or the nullptr keyword.

Explanation:

- In C++, a null pointer is a special pointer value that does not point to any valid object or memory
 - In older C++ code, `0` or `NULL` was used to represent a null pointer
 - Since C++11, the safer and preferred way is using `nullptr`
 - A null pointer is not the same as an uninitialized pointer — an uninitialized pointer contains an indeterminate (garbage) value
 - It doesn't "point to a memory address containing zeros"; it just holds a special address value indicating "no object"
-

Question 3: Const Keyword with Pointers

Question: What does the "const" keyword mean when used in the following pointer declaration?

```
cpp
```

```
const int* ptr;
```

Options:

- ptr is a constant pointer to an integer.
- ptr is a pointer to a constant integer
- ptr is a constant pointer to a constant integer
- None

Answer:  ptr is a pointer to a constant integer

Explanation:

- You cannot change the value being pointed to through `ptr` (`*ptr = 5;` is illegal)
 - But you can make `ptr` point to a different address
 - If it were `int* const ptr;`, then it would be a constant pointer (address cannot change)
 - If it were `const int* const ptr;`, then it would be a constant pointer to a constant integer
-

Question 4: Pointers vs References

Question: In C++, what is the difference between a pointer and a reference?

Options:

- Pointers can be reassigned to point to different objects, while references cannot.
- References can be reassigned to point to different objects, while pointers cannot.
- Pointers and references are exactly the same and can be used interchangeably.
- Pointers and references cannot be used in C++.

Answer:  Pointers can be reassigned to point to different objects, while references cannot.

Explanation:

- **Pointer:** Stores the address of a variable, can be reassigned to point to different variables, and can be `nullptr`. Needs dereferencing (`(*ptr)`) to access the value.
- **Reference:** An alias for an existing variable, must be initialized when created, cannot be changed to refer to another variable later, and cannot be `nullptr`.

Example:

```
cpp

int a = 10, b = 20;
int* p = &a; // pointer to a
p = &b;      // can point to b now

int& r = a;  // reference to a
// r = b;   // ❌ this assigns value of b to a, doesn't rebind the reference
```

Question 5: Array Modification Function

Question: What will be the output of the code?

```
cpp
```

```
void modifyArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] *= 2;
    }
}

int main() {
    int values[] = {1, 2, 3, 4, 5};
    modifyArray(values, 5);
    for (int i = 0; i < 5; i++) {
        cout << values[i] << " ";
    }
    return 0;
}
```

Options:

- 12345
- 2 4 6 8 10
- Runtime error
- Compilation Error

Answer:  2 4 6 8 10

Explanation:

- Arrays in C++ are passed to functions by pointer, so the function works directly on the original array
- `arr[i] *= 2;` doubles each element
- Initial array: {1, 2, 3, 4, 5}
- After modification: {2, 4, 6, 8, 10}

Question 6: Complex Pointer Operations

Question: What is the output of the following code?

```
cpp
```

```

#include <iostream>
void modifyValue(int* ptr) {
    (*ptr) *= 2;
}

void modifyArray(int* arr, int size) {
    for (int i = 0; i < size; i++) {
        modifyValue(&arr[i]);
    }
}

int main() {
    int x = 5;
    int* ptr = &x;
    int y = 10;
    int* ptr2 = &y;
    int** ptr_ptr = &ptr;
    int*** ptr_ptr_ptr = &ptr_ptr;

    modifyValue(*ptr_ptr);
    modifyValue(**ptr_ptr_ptr);
    modifyValue(ptr2);

    std::cout << "x: " << x << std::endl;
    std::cout << "y: " << y << std::endl;

    int array[] = {1, 2, 3, 4, 5};
    int* array_ptr = array;
    modifyArray(array_ptr, 5);
    std::cout << "Modified Array: ";
    for (int i = 0; i < 5; i++) {
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;
    return 0;
}

```

Answer:

```

x: 20
y: 20
Modified Array: 2 4 6 8 10

```

Explanation:

1. Modifying x:

- `modifyValue(*ptr_ptr)` → `*ptr_ptr` → `ptr` → `&x`, doubles `x` from 5 to 10
- `modifyValue(**ptr_ptr_ptr)` → `**ptr_ptr_ptr` → `ptr` → `&x`, doubles `x` from 10 to 20

2. Modifying y:

- `modifyValue(ptr2)` → `ptr2` → `&y`, doubles `y` from 10 to 20

3. Array modification: {1, 2, 3, 4, 5} → {2, 4, 6, 8, 10}

Question 7: Character and Integer Operations

Question: What will be the output of the following program?

```
cpp
#include <iostream>
using namespace std;

int main() {
    int a = 32, *ptr = &a;
    char ch = 'A', &cho = ch;
    cho += a;
    *ptr += ch;
    cout << a << "," << ch << endl;
    return 0;
}
```

Options:

- 32, A
- 32, a
- 129, a
- 129, A

Answer:  129, a

Explanation:

1. **Initial values:** `a = 32`, `ch = 'A'` (ASCII 65)

2. **Step 1:** `cho += a` → `ch = ch + a` → `65 + 32 = 97` → ASCII 97 = 'a'

3. **Step 2:** `*ptr += ch` → `a = a + ch` → `32 + 97 = 129`

Question 8: Const Pointer Declaration Error

Question: What will be the output of the following program?

```
cpp

#include <iostream>
using namespace std;

int main() {
    const int i = 20;
    const int const ptr = &i;
    (*ptr)++;
    int j = 15;
    ptr = &j;
    cout << i;
    return 0;
}
```

Options:

- 20
- 21
- 15
- Compilation Error

Answer:  Compilation Error

Explanation:

- `const int const ptr = &i;` is not a valid pointer declaration
- This declares `ptr` as a `const int`, not as a pointer
- Assigning `&i` (an address) to a `const int` is a type mismatch → Compilation Error
- Correct syntax would be: `const int* ptr = &i;` or `const int* const ptr = &i;`

Question 9: Pointer Arithmetic with Arrays

Question: What will be the output of the following program?

```
cpp
```

```
#include <iostream>
using namespace std;

int main() {
    int num[5];
    int* p;
    p = num;
    *p = 10;
    p++;
    *p = 20;
    p = &num[2];
    *p = 30;
    p = num + 3;
    *p = 40;
    p = num;
    *(p + 4) = 50;

    for (int i = 0; i < 5; i++)
        cout << num[i] << " ";
    return 0;
}
```

Options:

- 10, 20, 30, 40, 50
- 10, 20, 30, 40, 50,
- compiler error
- runtime error

Answer: ☒ 10, 20, 30, 40, 50,

Explanation:

1. $p = \text{num}; *p = 10;$ → $\text{num}[0] = 10$
2. $p++; *p = 20;$ → $\text{num}[1] = 20$
3. $p = \&\text{num}[2]; *p = 30;$ → $\text{num}[2] = 30$
4. $p = \text{num} + 3; *p = 40;$ → $\text{num}[3] = 40$
5. $p = \text{num}; *(p + 4) = 50;$ → $\text{num}[4] = 50$

Final array: {10, 20, 30, 40, 50} → Output: 10,20,30,40,50,

Question 10: Array Pointer Type Mismatch

Question: What will be the output of the following program?

```
cpp

#include <iostream>
using namespace std;

int main() {
    int arr[] = {4, 5, 6, 7};
    int p = (arr + 1);
    cout << *arr + 10;
    return 0;
}
```

Options:

- Error
- 12
- 15
- 14

Answer: ☒ Error (Compilation Error)

Explanation:

- `arr` decays to a pointer (`int*`) when used in expressions like `(arr + 1)`
- `(arr + 1)` is of type `int*` (pointer to `arr[1]`)
- But it's assigned to `int p` (plain integer), which is a type mismatch
- You can't directly store an address in an `int` without a cast
- If it were `int* p = (arr + 1);`, then `*arr + 10 = 4 + 10 = 14`

Question 11: Reference Declaration Error

Question: What will be the output of the following program?

```
cpp
```

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, *pa, &ra;
    pa = &a;
    ra = a;
    cout << "a=" << ra;
    return 0;
}
```

Options:

- 10
- Address
- compiler error
- runtime error

Answer:  Compilation Error

Explanation:

- In C++, references must be initialized at declaration
- You can't write: `int& ra; ra = a;` (This assigns value, not binding)
- Instead, you must do: `int& ra = a;` (bind reference to `a` at creation time)
- Since the given code does not initialize `ra` immediately, this is a compilation error

Question 12: Operator Precedence

Question: `ptrdata` is a pointer to a data type. The expression `*ptrdata++` is evaluated as:

Options:

- `*(ptrdata++)`
- `(*ptrdata)++`
- Depends on compiler
- `*(ptrdata)++`

Answer:  `*(ptrdata++)`

Explanation:

- In C++, postfix `++` (increment) has higher precedence than the dereference operator `*`
 - So `ptrdata++` happens before the `*` dereference
 - This means: use the value pointed to by `ptrdata` before incrementing the pointer
 - After this expression, `ptrdata` will point to the next element
-

Question 13: Function Pointer Declaration

Question: What does the following statement mean?

cpp

```
int (*fp)(char*)
```

Options:

- pointer to a pointer
- pointer to an array of chars
- pointer to function taking a char* argument and returns an int
- function taking a char* argument and returning a pointer to int

Answer: ☒ pointer to function taking a char* argument and returns an int

Explanation:

- `fp` is inside parentheses with `*` → means `fp` is a pointer
- `(char*)` after it → means it's a pointer to something that can be called like a function taking `char*` as a parameter
- `int` at the start → means that function returns an `int`

Example usage:

cpp

```
int myFunc(char* str) {  
    return strlen(str);  
}  
  
int (*fp)(char*) = myFunc; // fp points to myFunc  
cout << fp("Hello");      // calls myFunc("Hello")
```

Question 14: Pointer Equivalence

Question: The following two statements are equivalent to `n = *&q`. These statements are:

cpp

```
p = &q;
```

```
n = *p;
```

which are further equivalent to:

Options:

- `n = p`
- `p = q`
- `n = p*q`
- `n = q`

Answer:  `n = q`

Explanation:

- `n = *&q;` → `&q` gets address of `q`, `*&q` dereferences that address → gives the value of `q`
- So `n = *&q;` basically means `n = q;`
- Given: `p = &q; n = *p;` → `p` points to `q`, `n` gets value at address stored in `p` → value of `q`
- All are equivalent to `n = q;`

Question 15: Complex Array Pointer Arithmetic

Question: Consider the following code:

cpp

```
int main() {  
    int a[] = {2,4,6,8,10};  
    int i, sum=0, *b=a+4;  
    for(i=0; i<5; i++)  
        sum = sum + (*b-i) - *(b-i);  
    cout << sum << endl;  
}
```

Options:

- 10

- 20
- 15
- 30

Answer:  10

Explanation:

- $b = a + 4 \rightarrow b$ points to $a[4]$ (value 10)
- Loop iterations:
 - $i=0$: $(*b-i) - *(b-i) = (10-0) - 10 = 0$, $sum = 0$
 - $i=1$: $(10-1) - *(b-1) = 9 - 8 = 1$, $sum = 1$
 - $i=2$: $(10-2) - *(b-2) = 8 - 6 = 2$, $sum = 3$
 - $i=3$: $(10-3) - *(b-3) = 7 - 4 = 3$, $sum = 6$
 - $i=4$: $(10-4) - *(b-4) = 6 - 2 = 4$, $sum = 10$

Question 16: Pointer Swapping Function

Question: What is the output?

cpp

```
void rer(int *ptr2, int **ptr1) {
    int *ii;
    ii = ptr2;
    *ptr2 = *ptr1;
    *ptr1 = ii;
    **ptr1 *= **ptr2;
    **ptr2 += **ptr1;
}

int main() {
    int var1 = 5, var2 = 10;
    int *ptr1 = &var1, *ptr2 = &var2;
    rer(&ptr1, &ptr2);
    cout << var2 << " " << var1 << endl;
}
```

Options:

- 50 50

- 50 60
- 60 50
- 60 60

Answer:  60 50

Explanation: This is a complex pointer manipulation that swaps pointer targets and performs arithmetic:

1. Function swaps what `(ptr1)` and `(ptr2)` point to
 2. Multiplies `(var1)` by `(var2)`: `(5 * 10 = 50)`
 3. Adds result to `(var2)`: `(10 + 50 = 60)`
 4. Final: `(var1 = 50)`, `(var2 = 60)`
 5. Output: `(60 50)`
-

Question 17: Array of Pointers

Question: What is the output?

```
cpp
int main() {
    int a[] = {10,20,30,40,50};
    int *p[] = {a, a+3, a+4, a+1, a+2};
    int **ptr = p;
    ptr++;
    cout << ptr-p << **ptr;
}
```

Options:

- 140
- 140
- 140
- None

Answer:  140

Explanation:

1. **Array a:** `{10, 20, 30, 40, 50}`

2. Array of pointers p:

- $p[0] \rightarrow a$ (points to 10)
- $p[1] \rightarrow a+3$ (points to 40)
- $p[2] \rightarrow a+4$ (points to 50)
- $p[3] \rightarrow a+1$ (points to 20)
- $p[4] \rightarrow a+2$ (points to 30)

3. **ptr operations:** $ptr = p; ptr++;$ $\rightarrow$ ptr now points to $p[1]$

4. **ptr - p:** 1 (pointer arithmetic)

5. ****ptr:** $ptr \rightarrow p[1] \rightarrow a+3 \rightarrow 40$

6. **Output:** 1 followed by 40 = 140

Question 18: Pointer Size Comparison

Question: If X, Y and Z are pointer variables of type char, int and float, respectively in C++ language, then which of the following statements is true?

Options:

- Size of X, Y and Z are same
- Size of Z is greater than the size of X
- Size of Y is greater than the size of X
- Size of Z is greater than the size of Y

Answer: ☒ Size of X, Y and Z are same

Explanation:

- In C++, the size of a pointer does not depend on the type it points to
 - It depends on the platform/architecture (e.g., 4 bytes on 32-bit, 8 bytes on 64-bit)
 - $char^*$, int^* , and $float^*$ all have the same size on the same machine
 - A pointer just stores an address — the compiler needs to know the type only for dereferencing and pointer arithmetic, not for changing the pointer size
-

Question 19: Array Size Calculation

Question: Given in a system, s is the size of 1 integer, and 'arr' is an integer array, how can we calculate number of elements n in the array without using in-built functions?

Options:

- `n=(*(&a+1)-*(&a))/s;`
- `n = (*(&a+1)-*(&a));`
- `n = ((&a+1)-(&a));`
- None

Answer:  `n = ((&a+1)-(&a));`

Explanation:

- `&arr` = address of the whole array (type: pointer to array)
 - `&arr + 1` = address just past the entire array
 - `(&arr + 1) - (&arr)` gives the number of elements directly (no need to divide by size)
 - Pointer arithmetic automatically accounts for the element size
-

Question 20: Array Size Calculation Output

Question: What is the output?

cpp

```
int main() {  
    int a[] = {10,20,30,40,50,60};  
    cout << (*(&a+1)-*(&a));  
}
```

Options:

- Some address value
- 06
- 50
- 60

Answer:  6 (or 06)

Explanation:

1. **Understanding &a:** `&a` is the address of the whole array (type: `int (*)[6]`)
2. **&a + 1:** moves the pointer past the entire array (by $6 \times \text{sizeof(int)}$ bytes)
3. **Dereferencing:**

- $\text{*}(\&a + 1)$ → points to $a[6]$ (one past the last element)
- $\text{*}(\&a)$ → points to $a[0]$

4. **Subtraction:** $\text{*}(\&a + 1) - \text{*}(\&a) = (a + 6) - (a) = 6$

5. **Output:** 6

Memory visualization:

- Array occupies addresses 1000–1023 (assuming 4-byte ints)
 - $\&a$ points to 1000, $\&a + 1$ points to 1024
 - Pointer subtraction gives element count: 6
-

Summary

This collection covers essential C++ pointer concepts including:

- Basic pointer operations and relationships
- Const pointers and pointers to const
- Pointer vs reference differences
- Array and pointer arithmetic
- Function pointers
- Multi-level pointers (pointer to pointer)
- Memory addressing and size calculations
- Common compilation errors with pointers

These questions demonstrate the importance of understanding pointer arithmetic, operator precedence, type safety, and memory layout in C++ programming.